

Design a Chat System

WhatsApp · Telegram · Signal · Slack – "WebSocket తెరిచి messages పంపుదాం, timestamp తో క్రమంలో పెడదాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఫోన్ గడియారంతో క్రమం పెడితే **సగం జవాబులు ప్రశ్నకి ముందు** కనిపిస్తాయి. రెండోది – ఆ పచ్చ చుక్క (online) మీ నిజమైన messages కంటే **2,400 రెట్లు** traffic సృష్టిస్తుంది. మూడోది – 256 మంది గుంపులో ఒక్క message **511 సందేశాలు** అవుతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ DOC యొక్క ఒక్క వాక్య సారాంశం

Chat system design lo అందరూ **messages** గురించి మాట్లాడతారు – WebSocket, delivery, storage.

కానీ కొలిస్తే తేలేది ఇది:

మీ system మోసే traffic lo messages ఒక చిన్న భాగం మాత్రమే. మిగతాదంతా – presence, receipts, typing – "ఎవరు ఏమి చేస్తున్నారు" అనే మెటాడేటా.

§7 lo presence నిజమైన messages కంటే **2,400 రెట్లు**. §10 lo ఒక గుంపు message **511 రెట్లు** విస్తరిస్తుంది. రెండూ కలిపితే – మీ chat system యొక్క 99.9% పని **chat కాదు**.

మరియు §4 – క్రమం. అది traffic సమస్య కాదు, అది **సత్య** సమస్య: ఒక్క timestamp ఎంపిక సగం సంభాషణలని అర్థరహితం చేస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు నిజమైన భారం ఎక్కడ?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: క్రమం

3. Step – timestamp తో క్రమం పెట్టడం
4. **మొదటి విరుపు** – సగం జవాబులు ప్రశ్నకి ముందు
5. Step – సంభాషణకి ఒక వరుస సంఖ్య

Part 3 – రెండో విరుపు: పచ్చ చుక్క

6. Step – స్థితి మారితే contacts అందరికీ చెప్పడం
7. **రెండో విరుపు** – messages కంటే 2,400 రెట్లు

Part 4 – మూడో విరుపు: మూడు ticks

8. Step – ✓ · ✓✓ · నీలం ✓✓
9. **మూడో విరుపు** – ఒక message, 511 సందేశాలు
10. Step – కిటికీలో కలపడం

Part 5 – పూర్తి system

Part 6 – Interview lo

12. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
13. నోటీతో చెప్పాల్సిన English script
14. Follow-ups – encryption, బహుళ పరికరాలు, offline
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు నిజమైన భారం ఎక్కడ?

A, B కి ఒక సందేశం పంపుతాడు. అది B కి చేరాలి – B online ఉన్నా, లేకపోయినా. గుంపుల్లో అందరికీ చేరాలి. మరియు అందరికీ **ఒకే క్షణంలో** కనిపించాలి.

దీనికి WebSocket కావాలి (server నుంచి client కి *push* చేయాలి కాబట్టి, HTTP polling కాదు), ఒక message store కావాలి, ఒక delivery పొర కావాలి. అదంతా సూటిగా ఉంది.

కానీ ఒక లెక్క వేద్దాం. **10 లక్షల వాడుకరులు, ఒక్కొక్కరు రోజుకి 40 messages:**

నిజమైన chat messages : 463 / సెకను

సెకనుకి 463. అది **ఏమీ కాదు** – ఒక్క machine దాన్ని మోయగలదు.

అయితే WhatsApp కి వేల servers ఎందుకు? ఎందుకంటే ఆ 463 **మొత్తం traffic lo ఒక భిన్నం** మాత్రమే.

అదే 10 లక్షల వాడుకరులు · మిగతా TRAFFIC

presence (వచ్చు చుక్క) – సెకనుకి **11,11,111** notifications (§7)

receipts (✓✓ మరియు నీలం) – ఒక్కే గుంపు message కి **511** రెట్లు (§9)

చాట్ system యొక్క 99.9% పని chat కాదు.

అందుకే ఈ doc lo message storage గురించి ఒక్క విరుపు కూడా లేదు. మూడు విరుపులూ **messages చుట్టూ ఉన్న వాటి** గురించే.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
అందరికీ ఒక క్రమం కనిపించాలా?	అవును అంటే timestamps సరిపోవు – §5
Presence (online/last seen) కావాలా?	ఇది ఒక feature కాదు, ఇది మీ అతి పెద్ద traffic మూలం – §7
Read receipts (నీలం ticks) కావాలా?	గుంపుల్లో ఇది N రెట్లు . మరియు ఇది ఒక privacy నిర్ణయం కూడా
గుంపు గరిష్ఠ పరిమాణం ఎంత?	ఈ సంఖ్య మీ receipt design ని నిర్ణయిస్తుంది. 10 vs 256 అంటే 19x vs 511x
Messages శాశ్వతంగా నిల్వ చేయాలా?	చేయకపోతే (Signal వంటివి) storage problem దాదాపు పోతుంది
End-to-end encryption ఉందా?	ఉంటే server content చూడలేదు – search, spam filter అన్నీ client మీదికి వెళ్తాయి
ఒక వాడుకరికి ఎన్ని పరికరాలు?	ఒకటి కంటే ఎక్కువ అయితే delivery ఒక fan-out – §14

ఒక ప్రశ్న INTERVIEWER ని ఆశ్చర్యపరుస్తుంది

"Presence మరియు typing indicators scope lo ఉన్నాయా?"

ఎందుకంటే చాలామంది వాటిని చిన్న features అనుకుంటారు – "తర్వాత చేసుకుందాం". కానీ §7 lo కొలుస్తాం: **presence ఒక్కటే మీ message traffic కంటే 2,400 రెట్లు.**

అంటే presence **scope lo ఉందా లేదా** అనేది మీ మొత్తం capacity plan ని మారుస్తుంది. అది ముందే అడగాలి.

Part 2 – మొదటి విరుపు: క్రమం

3. Step – timestamp తో క్రమం పెట్టడం

సహజమైన design: ప్రతి message తో ఒక timestamp, మరియు గ్రాహకుడు దాని ప్రకారం క్రమంలో పెడతాడు.

```
// పంపేటప్పుడు
socket.send({ to: 'bob', text: 'హే', at: Date.now() });

// చూపించేటప్పుడు
messages.sort((a, b) => a.at - b.at);
```

`Date.now()` ఎవరి గడియారం? రెండు ఎంపికలు:

- **పంపినవాడి ఫోన్** – అతని గడియారం తప్పు కావచ్చు
- **Server** – message చేరిన క్షణం

రెండూ సహజంగా అనిపిస్తాయి. కొలుద్దాం.

4. మొదటి విరుపు – సగం జవాబులు ప్రశ్నకి ముందు

"క్రమం తప్పింది" అంటే ఏమిటో ముందు స్పష్టం చేయాలి. రెండు వేర్వేరు విషయాలు ఉన్నాయి:

- **ఏకరూపత** – అందరికీ ఒకే క్రమం కనిపించడం
- **కారణ-కార్యం** – B, A యొక్క message ని చూశాక జవాబు ఇస్తే, జవాబు ఎప్పుడూ తర్వాతే కనిపించడం

రెండోదే ముఖ్యమైనది, ఎందుకంటే అది విరిగితే **సంభాషణ అర్థరహితం** అవుతుంది. దాన్నే కొలిచాను:

200 × 500 ప్రశ్న-జవాబు జతలు • B ప్రతిసారీ ప్రశ్న *చూశాక* జవాబు ఇచ్చాడు

క్రమం ఎలా నిర్ణయిస్తున్నాం పరిస్థితి		జవాబు ముందు కనిపించినవి
ఫోన్ గడియారం	గడియారం తేడా 0.5s	27973 (28.0%)
ఫోన్ గడియారం	గడియారం తేడా 2s	42988 (43.0%)
ఫోన్ గడియారం	గడియారం తేడా 30s	49924 (49.9%)
server చేరిన సమయం	jitter 300ms	0 (0.0%)
server చేరిన సమయం	jitter 2000ms	0 (0.0%)
server వరుస సంఖ్య	-	0 (0.0%)

ఫోన్ గడియారం - అర్థభాగం జవాబులు ప్రశ్నకి ముందు

కేవలం అర సెకను గడియారం తేడాతో **28%**. 30 సెకన్లతో **49.9%** - అంటే నాణెం విసిరినట్టు.

మరియు ఫోన్ గడియారాలు నిజంగా తప్పుగా ఉంటాయి. వాడుకరి manual గా మార్చేస్తే, timezone మార్చేస్తే, NTP విఫలం కావచ్చు.

"పంపినవాడి timestamp" అనేది ఒక design ఎంపిక కాదు - అది ఒక bug.

కానీ ఆశ్చర్యం: **server timestamp 0% ఇచ్చింది.** ఎందుకు?

ఎందుకంటే జవాబు **భౌతికంగా** ప్రశ్న తర్వాతే server కి చేరుతుంది - B ముందు ప్రశ్న అందుకోవాలి, చదవాలి, తర్వాత జవాబు పంపాలి. Network jitter ఎంత ఉన్నా ఆ క్రమం మారదు.

అయితే server timestamp సరైనదేనా? - ఒక్క server ఉంటే.

అదే పరీక్ష - కానీ ఇద్దరూ *వేరే వేరే chat servers* కి కనెక్ట్ అయితే

servers మధ్య గడియారం తేడా జవాబు ముందు కనిపించినవి	
1 ms	0 (0.0%)
10 ms	0 (0.0%)
50 ms	275 (0.3%)
250 ms	12905 (12.9%)

250 MS SERVER గడియారం తేడా → 12.9% సంభాషణలు విరిగాయి

NTP బాగా పని చేస్తే servers మధ్య తేడా 1-10 ms, మరియు అక్కడ ఇది 0%. కానీ:

- ఒక VM live-migrate అయితే దాని గడియారం దూకుతుంది
- NTP విఫలమైతే drift పెరుగుతుంది (రేజికి కొన్ని వందల ms సాధారణం)
- Cloud lo ఒక noisy neighbour గడియారాన్ని ఆలస్యం చేస్తుంది

మీ సత్యం NTP మీద ఆధారపడకూడదు. మరియు ఇక్కడ అది ఆధారపడుతోంది.

5. Step - సంభాషణకి ఒక వరుస సంఖ్య

పరిష్కారం గడియారాన్ని పూర్తిగా వదిలేయడం. **ప్రతి సంభాషణకి ఒక యజమాని**, మరియు అతను వరుస సంఖ్యలు ఇస్తాడు:

```
// క్రమాన్ని *ఇక్కడ* నిర్ణయిస్తాం - గడియారంతో కాదు. §4 చూడండి.  
append(from, text, at) {  
  if (!this.members.has(from)) return { ok: false, reason: 'NOT_MEMBER' };  
  const m = { seq: ++this.#seq, from, text, at };  
  this.#msgs.push(m);
```

```
return { ok: true, seq: m.seq };  
}
```

at ఇంకా నిల్వ చేస్తాం – **చూపించడానికి** ("ఉదయం 10:32"). కానీ **క్రమానికి** వాడం. ఆ రెండు పనులని వేరు చేయడమే ఇక్కడి మొత్తం ఆలోచన.

"సంభాషణకి ఒక యజమాని" – ఇది ఒక పరిమితి లాగా అనిపిస్తుంది, కానీ అది ఒక బహుమతి

ఒక సంభాషణ యొక్క అన్ని messages **ఒకే shard** కి వెళ్తాయి (`hash(convId)`). అప్పుడు:

- వరుస సంఖ్య ఇవ్వడం ఒక local counter – ఏ coordination అవసరం లేదు
- ఆ సంభాషణ చదవడం ఒకే shard query
- **ఖాళీలు కనిపిస్తాయి** – client కి 1,2,3,5 వస్తే 4 పోయిందని వెంటనే తెలుసు, మరియు అది అడగగలదు

ఆ చివరి దాన్ని timestamp ఎప్పటికీ ఇవ్వలేదు. **Timestamp తో "ఒకటి పోయింది" అని మకు ఎప్పటికీ తెలియదు.**

ఒక message, మూడు వేర్వేరు "సత్యాలు"

ఫోన్ గడియారం
వాడుకరి మార్పుగలదు
49.9% తప్పు

server గడియారం
NTP మీద ఆధారపడుతుంది
12.9% తప్పు (250ms drift)

వరుస సంఖ్య
ఏ గడియారం మీదా ఆధారపడదు
0% తప్పు

Timestamp ఒక కొలత – అది ఎప్పుడూ ఉజ్జాయింపు.

వరుస సంఖ్య ఒక నిర్ణయం – అది ఖచ్చితంగా సరైనది, ఎందుకంటే అదే సత్యాన్ని నిర్ణయిస్తుంది.

సమయాన్ని **చూపించడానికి** వాడండి.

వరుస సంఖ్యని **క్రమం పెట్టడానికి** వాడండి.

ఆ రెండూ ఒకటే అనుకోవడమే 54 యొక్క తప్పు.

Part 3 – రెండో విరుపు: పచ్చ చుక్క

6. Step – స్థితి మారితే contacts అందరికీ చెప్పడం

Presence సులభంగా అనిపిస్తుంది: వాడుకరి online అయితే అతని contacts అందరికీ చెప్పాలి, offline అయినా అంతే.

```
function onStatusChange(user, status) {  
  for (const contact of contactsOf(user)) {  
    push(contact, { type: 'presence', user, status });  
  }  
}
```

ఇది **fan-out on write** – HLD Deep 03 §3 lo చూసిన అదే ఆకారం. మరియు అక్కడ నేర్చుకున్న పాఠం ఇక్కడా వర్తిస్తుంది: **ముందు పంపిణీని చూడాలి**.

కానీ ఇక్కడ ఒక కొత్త గుణకం ఉంది: **స్థితి ఎంత తరచుగా మారుతుంది?**

Post ఒక ఉద్దేశపూర్వక చర్య – రోజుకి ఒకటి రెండు. కానీ presence **అప్రయత్నంగా** మారుతుంది: ఫోన్ జేబులోకి వెళ్ళినప్పుడు, WiFi నుంచి mobile కి మారినప్పుడు, app ని మూసినప్పుడు, lift lo signal పోయినప్పుడు.

7. రెండో విరుపు – messages కంటే 2,400 రెట్లు

1 మిలియన్ users • సగటున 200 contacts

స్థితి ఎంత తరచుగా మారుతుంది | మార్పులు/గంట | notifications/సెకను

1 సార్లు/గంట	1 మి	55,556
6 సార్లు/గంట	6 మి	3,33,333
20 సార్లు/గంట	20 మి	11,11,111
60 సార్లు/గంట	60 మి	33,33,333

పోలిక – నిజమైన chat messages : 463 / సెకను
presence notifications : 11,11,111 / సెకను
→ presence traffic messages కంటే 2400 రెట్లు ఎక్కువ

2,400 రెట్లు - మరియు ఇది ఒక FEATURE, ఒక అవసరం కాదు

మీ system యొక్క అసలు పని - messages - సెకనుకి 463.

ఆ పచ్చ చుక్క - సెకనుకి 11,11,111.

అంటే మీ servers, మీ network, మీ bill lo 99.96% ఒక చుక్క కోసం.

మరియు గమనించండి: ఇది N² సమస్య. వాడుకరులు రెట్టింపు అయితే, ఒక్కొక్కరి contacts కూడా పెరుగుతాయి - కాబట్టి traffic నాలుగు రెట్లు.

పరిష్కారం - ఎవరు నిజంగా చూస్తున్నారు?

ఇక్కడ ఒక సాధారణ జ్ఞానం ఉంది: ఒక వాడుకరి తన 200 contacts యొక్క పచ్చ చుక్కని ఒకేసారి చూడడు. అతని తెరమీద ఒకేసారి 8 మంది ఉంటారు.

కాబట్టి push ని చూస్తున్నవాళ్ళకి మాత్రమే పరిమితం చేద్దాం:

```
// §7 - presence ని contacts అందరికీ కాకుండా చూస్తున్నవాళ్ళకి మాత్రమే
watch(user, targets) { this.#watching.set(user, new Set(targets)); }

onStatusChange(user, contacts) {
  let pushed = 0;
  for (const c of contacts) {
    if (this.#watching.get(c)?.has(user)) { pushed++; this.stats.presencePush++; }
    else this.stats.presenceSkipped++;
  }
  return pushed;
}
```

పరిష్కారం: చూస్తున్న వాళ్ళకి మాత్రమే • ఒక్కో వాడుకరి తెరమీద సగటున 8 మంది

విధానం	notifications/సెకను	తగ్గింపు
contacts అందరికీ push	11,11,111	-
తెరమీద ఉన్నవాళ్ళకి మాత్రమే	44,444	96%

ఇది HLD DEEP 03 §8 యొక్క అదే ఆలోచన, వేరే వేషంలో

అక్కడ: చురుకుగా లేని వాళ్ళకి push చెయ్యకు (వృథా = 1 - DAU).

ఇక్కడ: చూడని వాళ్ళకి push చెయ్యకు (వృథా = 1 - 8/200).

రెండింటిలోనూ ఒకే ప్రశ్న: "నేను పంపుతున్న దీన్ని ఎవరైనా నిజంగా చూస్తారా?"

మరియు రెండింటిలోనూ జవాబు ఆశ్చర్యకరంగా "చాలావరకు లేదు".

మరియు ఒక ఉచిత లాభం – ఇది ఒక PRIVACY మెరుగుదల కూడా

"అందరికీ push" అంటే మీ 200 contacts lo **ఎవరైనా** మీరు ఎప్పుడు online అయ్యారో, ఎప్పుడు వెళ్ళిపోయారో **నిరంతరం** తెలుసుకోగలరు. దాన్ని నమోదు చేస్తే మీ రోజువారీ దినచర్య మొత్తం తెలుస్తుంది.

"చూస్తున్నవాళ్ళకి మాత్రమే" అంటే ఆ సమాచారం **మీ chat ని తెరిచి చూస్తున్నప్పుడు** మాత్రమే వెళ్తుంది.

ఒకే మార్పు – 96% traffic ఆదా, మరియు ఒక నిఘా దారి మూసివేత.

Part 4 – మూడో విరుపు: మూడు ticks

8. Step – ✓ • ✓✓ • నీలం ✓✓

WhatsApp యొక్క మూడు ticks:

- ✓ – server కి చేరింది
- ✓✓ – గ్రాహకుడి పరికరానికి చేరింది
- నీలం ✓✓ – గ్రాహకుడు చదివాడు

1:1 సంభాషణలో ఇది సులభం – ఒక message కి రెండు అదనపు సందేశాలు (delivered, read).

గుంపులో? ప్రతి సభ్యుడూ తన సొంత receipt పంపాలి.

9. మూడో విరుపు – ఒక message, 511 సందేశాలు

ఒక్క నిజమైన message కి system ఎన్ని సందేశాలు మోయాలి?

గుంపు పరిమాణం	నిజమైన messages	✓✓ receipts	నీలం receipts	మొత్తం	రెట్లు
2	1000	1000	1000	3000	3×
10	1000	9000	9000	19000	19×
50	1000	49000	49000	99000	99×
256	1000	255000	255000	511000	511×

256 మంది గుంపు – ఒక MESSAGE, 511 సందేశాలు

ఒక వ్యక్తి "సరే" అని రాస్తాడు. మీ system **510 అదనపు సందేశాలు** మోస్తుంది – 255 "చేరింది", 255 "చదివారు".

మరియు ప్రతి receipt మిగతా 255 మందికీ చేరాలి (ticks అందరికీ కనిపించాలంటే). అప్పుడు అది 511 కాదు, ఇంకా ఎక్కువ.

ఒక్క గుంపు, రోజుకి 500 messages → 2,55,500 సందేశాలు.

10. Step – కిటికీలో కలపడం

ముఖ్యమైన గమనిక: **receipts సంచితమైనవి (cumulative)**. "నేను 50వ message వరకు చదివాను" అనేది 1 నుంచి 50 వరకు అన్నిటికీ కవర్ చేస్తుంది.

అంటే **50 receipts కీ బదులు ఒక్కటి చాలు.**

```
// $10 - receipts ని వెంటనే పంపకుండా ఒక కిటికీలో కలపడం
queueAck(convId, user, upTo, kind) {
  const k = `${convId}|${user}|${kind}`;
  const cur = this.#pendingAcks.get(k);
  if (cur !== undefined && cur >= upTo) { this.stats.acksDropped++; return false; }
  this.#pendingAcks.set(k, upTo); // కొత్తది పాతదాన్ని మింగేస్తుంది
  this.stats.acksQueued++;
  return true;
}
```

ఆ `cur >= upTo` పంక్తి ముఖ్యం: ఒక కిటికీలో ఒకే వాడుకరి నుంచి 20 receipts వస్తే, **అత్యధిక సంఖ్య ఒక్కటే** మిగులుతుంది.

పరిష్కారాలు · 256 మంది గుంపు · 500 messages

విధానం	మొత్తం సందేశాలు	తగ్గింపు
అన్నీ · వెంటనే	2,55,500	—
నీలం ticks లేకుండా	1,28,000	50%
2 సెకన్ల కిటికీలో కలిపి	6,875	97%
రెండు	3,688	99%

2 సెకన్ల కిటికీ - 97% తగ్గింపు, మరియు వాడుకరికి ఏమీ తెలియదు

Tick 2 సెకన్ల ఆలస్యంగా నీలం అవడం ఎవరూ గమనించరు. కానీ మే system **2,55,500 నుంచి 6,875** సందేశాలకి దిగుతుంది.

నీలం ticks పూర్తిగా తీసేయడం (50%) కంటే **కలపడం (97%)** చాలా మెరుగు - మరియు అది ఒక feature ని కూడా తీసేయదు.

ఒక feature ని చంపడం కంటే దాని తరచుదనాన్ని తగ్గించడం దాదాపు ఎప్పుడూ మంచిది.

Part 5 – పూర్తి system

11. మొత్తం code · 4.3 లక్షల messages · mutation testing

నాలుగు నియమాలు

#	నియమం	విరిగితే అర్థం
1	seq 1 నుంచి, ఖాళీలు లేకుండా పెరగాలి	క్రమం యజమాని విరిగింది
2	ఒకే message ఎప్పుడూ ఒకే seq · మారదు	నిల్వ విరిగింది
3	Receipts ఎప్పుడూ ముందుకే కదులుతాయి	పాత receipt కొత్తదాన్ని వెనక్కి లాగింది
4	చదివాడంటే చేరినట్టే (read ≤ delivered)	రెండు స్థితులు విడిపోయాయి

నియమం 4 సూక్ష్మమైనది: "చదివాను" అనే receipt "చేరింది" కంటే ముందు రావచ్చు (network క్రమం మారితే). అప్పుడు `read > delivered` అవుతుంది – మరియు UI "చదివారు కానీ చేరలేదు" అని చూపిస్తుంది.

```
// చదివాడంటే చేరినట్టే – ఈ రెండూ ఎప్పుడూ కలిసి ఉండాలి
if (kind === 'read' && upTo > this.delivered.get(user))
  this.delivered.set(user, upTo);
```

1,200 యాదృచ్ఛిక ప్రయోగాలు · 4,32,291 messages
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

sent	4,32,291
acksQueued	1,37,147
acksFlushed	71,714
acksDropped	6,687
presencePush	2,20,898
presenceSkipped	6,06,528

ఆ చివరి రెండు వరుసలు §7 ని ధృవీకరిస్తున్నాయి: **8,27,426 presence సంఘటనల్లో 6,06,528 (73%) దాటేశాం** – ఎవరూ చూడటం లేదు కాబట్టి.

Test విఫలం కాగలదా? – ఐదు మార్పులు

మార్పు లేని code → 0/1200 విఫలం

receipt పరిధి పరీక్ష తీసేస్తే → 156/400 విఫలం
ఉదా: చివర 2: delivered 364 > 363

పాత receipt ని వదిలేయకపోతే → 400/400 విఫలం
ఉదా: 2: delivered వెనక్కి పోయింది 34→31
read ⇒ delivered ని అమలు చెయ్యకపోతే → 400/400 విఫలం
ఉదా: 0: read 11 > delivered 3
batch lo పాత ack కొత్తదాన్ని మింగితే → 0/400 విఫలం
seq ని గడియారంతో ఇస్తే → 400/400 విఫలం

చివరి వరుస **§4 కి రుజువు**: seq స్థానంలో timestamp పెడితే 400/400 విఫలం.

నాలుగో మార్పు పట్టుబడలేదు - 0/400 - మరియు అది సరైనదే

"Batch lo పాత ack ని ఉంచు, కొత్తదాన్ని వదిలేయ్" అని మార్చాను. అది ఏ నియమాన్నీ విరగ్గొట్టదు: receipts ఇంకా ముందుకే కదులుతాయి, పరిధి దాటవు, $read \leq delivered$ ఉంటుంది.

అది కేవలం పాత సమాచారం ఇస్తుంది – tick ఆలస్యంగా నీలం అవుతుంది.

అంటే అది ఒక తాజాదన లక్షణం, సరి-తప్పు లక్షణం కాదు. దాన్ని నియమాలతో పరీక్షించలేం; దాన్ని **§10 పట్టికతో** పరీక్షిస్తాం.

LLD Deep 25 lo backoff కి ఇదే జరిగింది. రెండు రకాల లక్షణాలకి రెండు రకాల రుజువులు.

Part 6 – Interview lo

12. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	"463 messages/సెకను" లెక్క వేసి, "ఇది సమస్య కాదు" అని చెప్పండి. తర్వాత presence సంఖ్య
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "presence scope lo ఉందా?" మరియు "గుంపు గరిష్ఠం ఎంత?"
9-14	WebSocket, connection registry, message store – త్వరగా
14-24	మొదటి విరుపు. ఏకరూపత vs కారణ-కార్యం వేరు చేసి, 49.9% చెప్పండి. తర్వాత multi-server 12.9%
24-32	రెండో విరుపు. Presence 2,400x. "చూస్తున్నవాళ్ళకి మాత్రమే" → 96%, మరియు privacy లాభం
32-40	మూడో విరుపు. 256 గుంపు → 511x. Receipts సంచితమైనవి → కలపడం → 97%
40-45	నాలుగు నియమాలు, ముఖ్యంగా <code>read ≤ delivered</code>

ఏమి తప్పక చెప్పాలి

1. Chat system యొక్క traffic lo chat ఒక చిన్న భాగం. 463 vs 11,11,111.
2. Timestamp ఒక కీలక; వరుస సంఖ్య ఒక నిర్ణయం. సమయాన్ని చూపించడానికి, seq ని క్రమం పెట్టడానికి.
3. Server timestamp కూడా సరిపోదు – NTP drift 250 ms అయితే 12.9%.
4. Presence ఒక feature కాదు, ఒక capacity నిర్ణయం.
5. Receipts సంచితమైనవి – అందుకే వాటిని కలపాల్సి, మరియు 97% ఆదా అవుతుంది.

ఏమి వదిలేయాలి

- WebSocket protocol వివరాలు – "sticky routing తో connection registry" అని ఒక వాక్యం
- Message storage schema – Cassandra/HBase అని చెప్పి ముందుకి
- Push notification (APNS/FCM) వివరాలు – అడిగితే మాత్రమే
- Typing indicators – presence యొక్క అదే ఆకారం, ఒక వాక్యంలో ముగించండి

13. నీటితో చెప్పాల్సిన English script

"Let me start with a number that reframes this problem. A million users sending forty messages a day is **463 messages per second**. That's nothing – one machine handles it. So if chat messages aren't the load, what is? Presence and receipts. I'll show that presence alone is about **2,400 times** the message traffic."

"First, ordering, because that's a correctness problem rather than a scale one. I want to separate two things: **consistency** – everyone sees the same order – and **causality** – if I reply to your message after reading it, my reply never appears above yours. Causality is the one that matters, because breaking it makes conversations nonsense."

"Ordering by the sender's phone clock fails badly. With half a second of clock skew, **28% of replies appear before the question they answer**; at thirty seconds it's **49.9%** – a coin flip. And phone clocks really are wrong; users change them manually."

"Server receive-time actually measures **0%** violations, which surprised me – a reply physically can't reach the server before the message it answers, because the replier had to receive it first. But that's with **one** server. With two servers whose clocks differ by 250 milliseconds, it goes to **12.9%**. NTP usually keeps you within ten milliseconds, but a live-migrated VM jumps, and NTP does fail. **I don't want my ordering to depend on NTP.**"

"So: a per-conversation **sequence number**, assigned by whoever owns that conversation. I still store the timestamp – for displaying '10:32' – but I never sort by it. A timestamp is a **measurement**; a sequence number is a **decision**, and it's exactly right because it defines the truth rather than estimating it. It also gives you something a timestamp never can: if a client receives 1, 2, 3, 5, it **knows** 4 is missing and can ask for it."

"Second, presence. Status changes aren't deliberate – the phone goes in a pocket, WiFi hands off, the app backgrounds. At twenty changes an hour with 200 contacts, that's **1.1 million notifications per second** against 463 real messages. It's also quadratic: double the users and contacts grow too, so traffic quadruples."

"The fix is noticing that nobody looks at 200 green dots at once – maybe eight are on screen. Push presence only to people **currently viewing you**: a **96% reduction**. And it's a privacy improvement for free, because otherwise any of your 200 contacts can log exactly when you're awake."

"Third, receipts. In a 256-person group, one message produces 255 delivered receipts and 255 read receipts – **511x amplification**. One group with 500 messages a day is 255,500 system messages. But receipts are **cumulative**: 'I've read up to 50' covers everything before it. So I coalesce them in a two-second window, keeping only the highest per user. That's a **97% reduction**, and nobody notices a tick turning blue two seconds later. Dropping read receipts entirely only saves 50% – **reducing a feature's frequency beats removing it**."

"Four invariants, 1,200 randomized runs, 432,000 messages, zero violations. The one I'd highlight is **read ≤ delivered** – a read receipt can arrive before the delivered receipt if the network reorders them, and then the UI claims someone read a message that never reached them. Mutation testing caught four of five; the one that survives is coalescing keeping the stale receipt instead of the fresh one, which breaks no invariant – it's a **freshness** property, so its evidence is the measurement table, not the fuzz."

14. Follow-ups – encryption, బహుళ పరికరాలు, offline

"End-to-end encryption ఉంటే ఏమి మారుతుంది?"

Server కి **content కనిపించదు** – కాబట్టి server-side search, spam filtering, media transcoding అన్నీ అసాధ్యం. కానీ §4, §7, §9 లో ఏదీ మారదు: **వరుస సంఖ్య, presence, receipts అన్నీ మెటాడేటా**, మరియు అవి encrypt కావు. అదే E2E encryption యొక్క పరిమితి – **ఎవరు ఎవరికి ఎప్పుడు రాశారో server కి తెలుసు**, ఏమి రాశారో తెలియదు. గుంపు keys పంపిణీ ఒక ప్రత్యేక problem (Signal యొక్క Sender Keys).

"ఒక వాడుకరికి 4 పరికరాలు ఉంటే?"

Delivery ఒక fan-out అవుతుంది: ఒక message → 4 sockets. మరియు **receipts ఒక కొత్త ప్రశ్న అడుగుతాయి** – నాలుగు పరికరాల్లో ఒకటి చదివితే "చదివారు" అనాలా? సాధారణ జవాబు అవును, కానీ అప్పుడు `read` ఒక్కో వాడుకరికి, ఒక్కో పరికరానికి కాదు – నా code లో `read` Map వాడుకరి కీతో ఉంది, అదే కారణం. Sync కోసం ప్రతి పరికరం తన సొంత `delivered` watermark ఉంచుకోవాలి.

"గ్రూపుడు offline ఉంటే?"

Message ఇప్పటికే store అయి ఉంది (§5), కాబట్టి "పంపడం" అనేది ఒక **pull** మాత్రమే – అతను తిరిగి వచ్చినప్పుడు `since(lastSeq)` అడుగుతాడు. ఇదే వరుస సంఖ్య యొక్క ఇంకొక లాభం: **"నేను 47 వరకు**

చూశాను, తర్వాతివి ఇవ్వు" అనేది ఒక ఖచ్చితమైన ప్రశ్న. Timestamp తో అది "సుమారు 10:32 తర్వాతివి" అవుతుంది, మరియు అంచుల్లో messages పోతాయి లేదా రెట్టింపు అవుతాయి.

"Typing indicator?"

Presence యొక్క అదే ఆకారం, కానీ ఇంకా తరచుగా (ప్రతి అక్షరానికీ). రెండు నియమాలు: (1) దాన్ని ఎప్పుడూ నిల్వ చేయకూడదు – అది ఒక క్షణిక సంకేతం, (2) దాన్ని rate-limit చేయాలి – 3 సెకన్లకి ఒకసారి చాలు. మరియు §7 లాగే, సంభాషణ తెరిచి ఉన్నవాళ్ళకి మాత్రమే.

"ఒక సంభాషణ చాలా పెద్దదైతే (10,000 messages)?"

since(seq, limit) ఇప్పటికీ pagination – కానీ నిల్వ ఒక shard మీద పెరుగుతూ పోతుంది. పరిష్కారం: సంభాషణని కాలం ప్రకారం విభజించడం (నెలకి ఒక partition), మరియు seq ని partitions అంతటా కొనసాగించడం. seq ఒక్కటే సంభాషణ యజమాని దగ్గర ఉంటుంది కాబట్టి అది సులభం.

15. ఏమి నేర్చుకున్నాం

1. Chat system యొక్క traffic lo chat ఒక చిన్న భాగం. 463 messages/సెకను vs 11,11,111 presence notifications/సెకను. మీ capacity plan messages మీద కాదు, మెటాడేటా మీద ఆధారపడుతుంది.

2. Timestamp ఒక కొలత; వరుస సంఖ్య ఒక నిర్ణయం. కొలత ఎప్పుడూ ఉజ్జాయింపు – ఫోన్ గడియారంతో 49.9% తప్పు, server గడియారంతో (250 ms drift) 12.9%. నిర్ణయం ఖచ్చితమైనది, ఎందుకంటే అదే సత్యాన్ని నిర్వచిస్తుంది.

3. "ఏకరూపత" మరియు "కారణ-కార్యం" వేర్వేరు లక్షణాలు. రెండింటినీ వేరుగా అడగాలి, మరియు రెండో దాన్ని విరగొట్టే design సంభాషణని అర్థరహితం చేస్తుంది.

4. Server timestamp కూడా ఒక బాహ్య ఆధారం. అది NTP మీద ఆధారపడుతుంది, మరియు NTP విఫలమవుతుంది. మీ సత్యం మీ అదుపులో ఉండాలి.

5. "ఎవరైనా దీన్ని నిజంగా చూస్తారా?" అనే ప్రశ్న పదేపదే 90%+ ఆదా చేస్తుంది. HLD Deep 03 lo inactive users (90%), ఇక్కడ presence (96%). రెండూ ఒకే ప్రశ్న.

6. ఒక feature ని చంపడం కంటే దాని తరచుదనాన్ని తగ్గించడం మంచిది. నీలం ticks పూర్తిగా తీసేస్తే 50% ఆదా; వాటిని 2 సెకన్లలో కలిపితే 97% – మరియు feature అలాగే ఉంటుంది.

7. సంచిత సంకేతాలని కలపొచ్చు. "50 వరకు చదివాను" అనేది 50 సందేశాలని ఒక్కదానిలో మోస్తుంది. మీ receipts సంచితమైనవి కాకపోతే, మీరు design ని తప్పుగా చేశారు.

8. కొన్ని మార్పులు నియమాలతో పట్టుబడవు, మరియు అది సరైనదే. Batch lo పాత receipt ఉంచడం 0/400 – అది ఒక తాజాదన లక్షణం. దాని రుజువు §10 పట్టిక, fuzz కాదు.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Chat system design అంటే messages ని రవాణా చేయడం కాదు – "ఎవరు ఎక్కడ ఉన్నారు, ఏమి చూశారు" అనే నిరంతర ప్రవాహాన్ని నియంత్రించడం.

Messages ఒక రోజుకి 40. Presence ఒక గంటకి 20×200 . Receipts ఒక message కి 511.

మీ design యొక్క ప్రతి కస్టమైన నిర్ణయం – \$4 lo క్లమం, \$7 lo presence, \$10 lo receipts – అన్నీ **ఆ ప్రవాహం గురించే**, messages గురించి కాదు.